Orientation Tracking and Panorama Reconstruction

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—Motion models using IMUs and visual data are very common, however, finding an efficient way to create such a model is still an engaging task. This report details the methods used to track the orientation of a body using gradient descent instead of probabilistic methods and reporting the performance compared to ground truth measurements provided by an encoder. This work is then applied for the application of panorama reconstruction, by timestamp matching with camera frames to stitch together images taken at various times and poses into one image.**

***Index Terms*—IMU, Motion Model, Orientation estimation, Gradient Descent, Encoder, Panorama, Reconstruction.**

I. Introduction

Although Inertial Measurement Units (IMUs) are an effective tool to determine object orientation, there exist inherent biases in the hardware that result in an error which we can measure.

Also, since IMUs have multiple sensors, being the gyroscope and the accelerometer, we can use outputs from both to determine a more accurate estimation of our orientation. This estimation has been achieved in multiple ways, using probabilistic methods(link paper here), filtering, and other sensor fusion methods.

In this paper, we will be implementing and benchmarking the effectiveness of using gradient descent as a solution to the problem of accurate orientation estimation. We perform Gradient Descent manually, by computing the gradient of our data with respect to a loss function and then modifying our estimates accordingly.

After the estimation of orientation and measuring the performance of Gradient Descent, we will also be showcasing its application in the formation of panoramic photos, by constructing a setup involving a camera and an IMU in a fixed pose with respect to each other, and timestamp matching to estimate the camera orientation using the estimated IMU orientation. We will then stitch the images together by converting the camera pose to the IMU pose, and then to the world pose to get an accurate panorama.

II. Problem Formulation

Before estimating orientation, we need to calculate the biases, which are estimated as follows:

$$accel_bias = accel_values - (([0,0,1])/accel_sens)$$
$$gyro_bias = gyro_values - (([0,0,0])/gyro_sens)$$

Where $accel_values$ and $gyro_values$ are data from the IMU, and $accel_sens$ and $gyro_sens$ are constants from the IMU datasheet. We calculate our biases on stationary IMU values to find the relevant IMU values in gyroscopic units of rad/sec and acceleration units of gravity($9.8m/s^2$). We can then convert our measured values into real-world values using these biases and sensitivities.

For the first problem of finding our orientation using the IMU data, we consider the following motion model to predict our dynamics:

$$q_{t+1} = f(q_t, \tau_t\omega_t) = q_t exp([0, \tau_t\omega_t/2])$$

However, only using data from the gyroscope is not good enough to develop an accurate model. Thus, we must use an observation model using our accelerometer data to compare our motion model against:

$$[0, a_t] = h(q_t) = q_t^{-1}[0,0,0,1]q_t$$

Finally, we can develop a cost function for gradient descent using both models:

$$c(q_{1:T}) =$$
$$1/2\Sigma_{t=0}^{T-1}||2log(q_{t+1}^{-1}f(q_t, \tau_t\omega_t))||_2^2 +$$
$$1/2\Sigma_{t=1}^{T}||[0, a_t] - h(q_t)||_2^2$$

To create the panorama, we do not solve any optimization problem, but we match our camera timestamp to the closest previous imu data timestamp to get a close estimate of our pose at the time of the photo being taken.

III. Methods and Approach

A. IMU Bias Estimation

Before we conduct any high level processing, we need to find the intrinsic IMU biases to measurements, and normalize our values according to the sensitivity and biases. To do this, we took the first 5 seconds of IMU data, which was static for all test and training sets, and used our formula derived in the previous section to estimate the biases and sensitivity, which was then averaged over the entire 5s window. This resulted in our values being normalized and consistent, rather than non-representative of real values.

B. Gradient Descent

Our core insight for orientation estimation is to manually find the gradient of our data using PyTorch, and using that gradient to perform GD over all our data. Although slower than SGD, due to the size of our data, this allows us to build

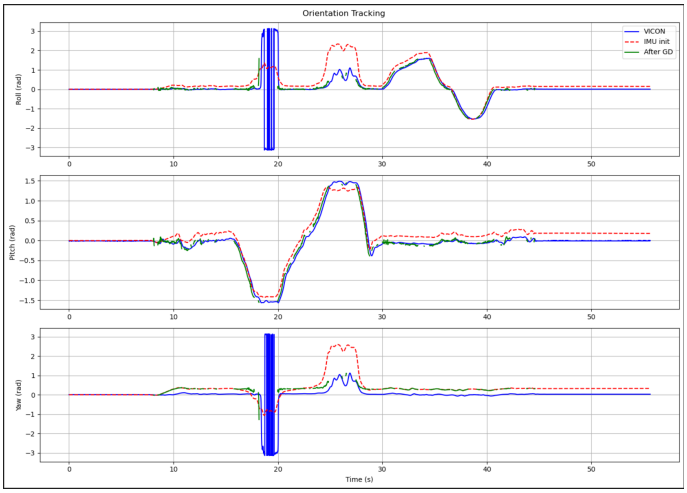


Fig. 1. Training gradient descent parameters

a more comprehensive picture of our data and perform more accurate gradient descent.

The algorithm we use is as follows:

Gradient Descent Algorithm

for n iterations:

$J = calculate_total_cost(quat)$

$gradient = jacobian(J, quat)$

$quat = quat - learning_rate * gradient$

Where n is the number of iterations, J is our loss for any iteration, $jacobian$ is the PyTorch implementation of a jacobian (torch.autograd.functional.jacobian), $quat$ is the list of all our quaternions calculated from the IMU, and $learning_rate$ is our learning rate for gradient descent.

With this algorithm, we can estimate the closest orientation to our ground truth VICON data.

In the Fig. 1, you can see the comparison of our ground truth data, IMU data pre gradient descent, and IMU data post gradient descent.

As is evident, our gradient descent algorithm, after fine tuning of the learning rate and number of iterations, performs very well at estimating an accurate pose for our body as compared to the ground truth VICON data.

C. Panorama Formation

For creating the panorama, we took our estimated orientation quaternions, and converted them to a rotation matrix, which will convert the imu frame to the world frame. We then calculated the fixed rotation matrix to change our images from the camera frame to the imu frame. Finally, we also take each pixel of our image and project it onto a sphere with field of view of $60°$ horizontally and $45°$ vertically.

Now, with our project in the world frame, we can project our image from spherical coordinates onto a cylinder, to then flatten out the cylinder at a yaw of $+-180°$ and show the image as part of our panorama.

These transformations, taken over every image which has been timestamp matched with a corresponding imu orientation, will give us a relatively smooth panorama of our image.

Panorama Algorithm

for k images:

$R = imu_quat_to_rot(q[t])$ (timestamp matched)

$rays = image_to_spherical(image_k_pixels)$

$rays_{world} =_{world} R_{cam} @_{imu} R_{cam} @ rays)$

$x, y = spherical_to_cylindrical(rays_{world}, pano_size)$

$panorama[x, y] = image_k$

$show\ panorama$

IV. Results

We will be evaluating our results for not only the panorama formed from the ground truth data, but comparing it to our imu estimation panorama, and finally showing only the performance on the test set.

We compare the results of two separate training datasets, as well as both test sets. Our goal is to illustrate how simple tools such as gradient descent and timestamp matching can lead to results that are satisfactory while having a low computational and time overhead.

A. IMU Orientation results

We used our algorithm as mentioned in the previous section, and tuned our parameters, arriving at a learning rate of 0.005, and 50 iterations of gradient descent to reach a satisfactory result.

1) Training Data: Using training sets 1 and 2, which we chose due to the availability of a camera stream along with them, we can compare our results from gradient descent with the ground truth values from the VICON measurements.

The remainder of the IMU data after gradient descent from training sets 3 through 9 are in Appendix A.

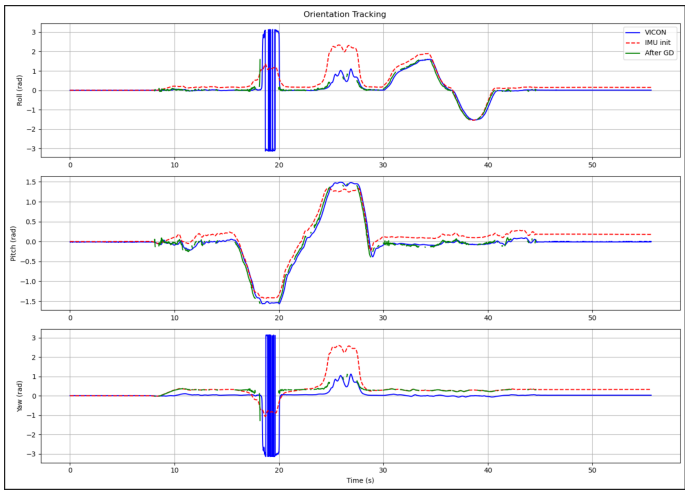


Fig. 2. Training gradient descent parameters for training set 1, and comparing the results with the ground truth orientations

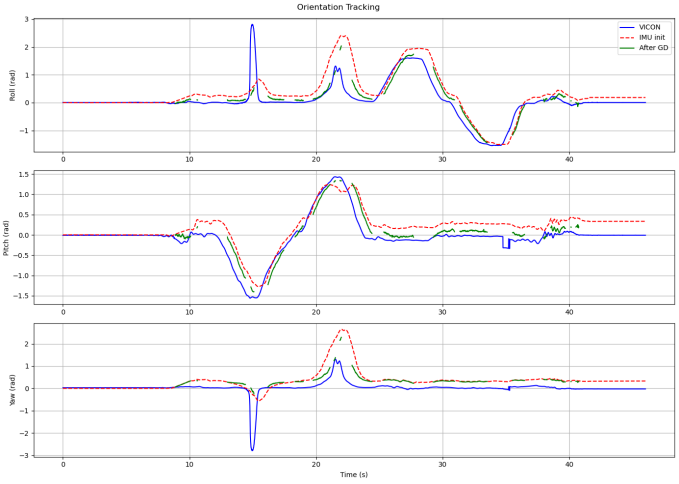


Fig. 3. Training gradient descent parameters for training set 2, and comparing the results with the ground truth orientations

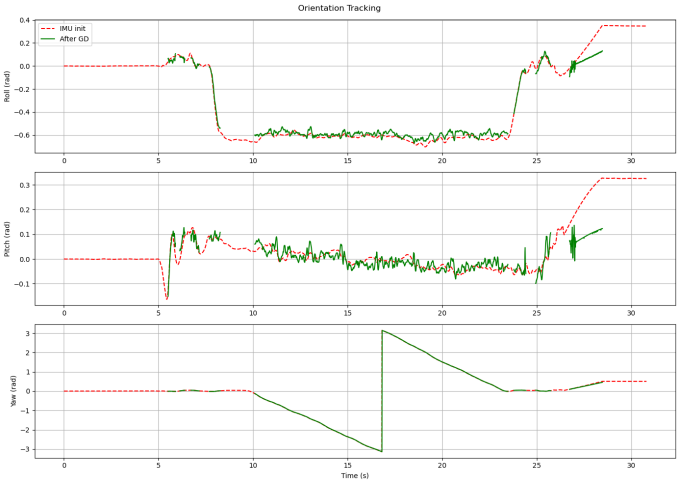


Fig. 4. Displaying the orientations after gradient descent compared to the initial IMU values in test set 10

As we can see from Fig. 2 and 3, our results are very similar when it comes to pitch and roll, since the effect of gravity on the body changes along with changes in the pitch and roll. However, we were unable to achieve similar results with yaw, since the gravity vector does not change with a change in the yaw. However, we can also see that the yaw does not drift significantly throughout the sampling period, so we can handle the minor error in the following task.

2) Test Data: In Fig. 4 and 5, we can see the orientations found after performing gradient descent on the IMU values from test sets 10 and 11 respectively. We see that there are not very significant changes to our data, either implying that our gradient descent was not very successful, or that IMU values are very similar to the ground truth values. The latter assumption is more believable due to results seen from the training data.

We also see that many values go to *nan* after gradient descent, and this could be due to the epsilon chosen for normalization being too large, or due to a myriad of other reasons. This is a limitation of the current implementation of gradient descent that would need to be improved in future work.

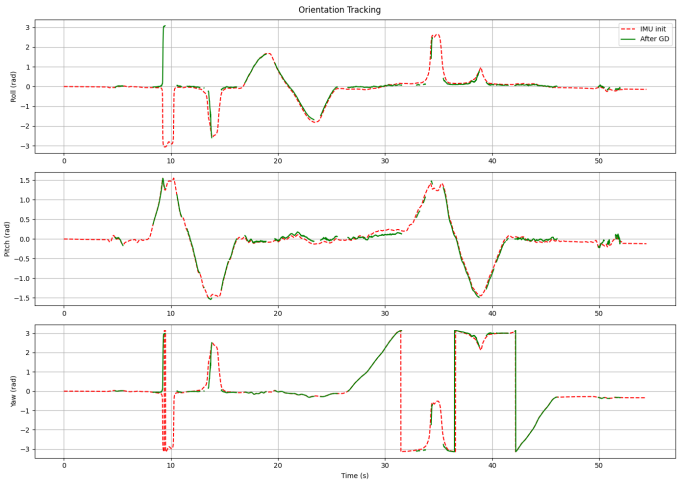


Fig. 5. Displaying the orientations after gradient descent compared to the initial IMU values in test set 11

B. Panorama comparison

We used the available camera data, which has its own timestamps, to match the orientation with an image taken at that orientation. Since the camera used is a global shutter camera with a fixed orientation relative to the IMU, we are able to make one single consistent transformation to give us an accurate panorama representation.

1) Training Data: In Fig. 6 and 7, we also used training sets 1 and 2, and are able to compare the panorama from our processed IMU data with the ground truth VICON data.

The remainder of the IMU data after gradient descent from training sets 8 and 9 are in Appendix B.

We can easily see that the images formed are quite similar, which matches the expectation since the orientations are very

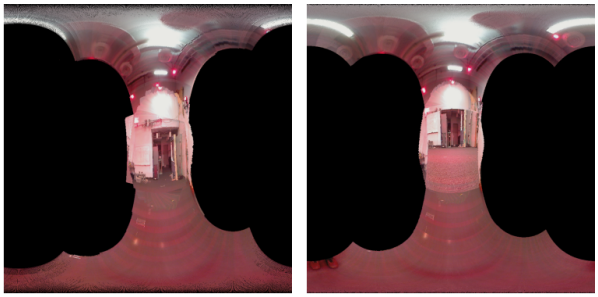


Fig. 6. Comparing the IMU formed panorama with the VICON ground truth panorama for training set 1

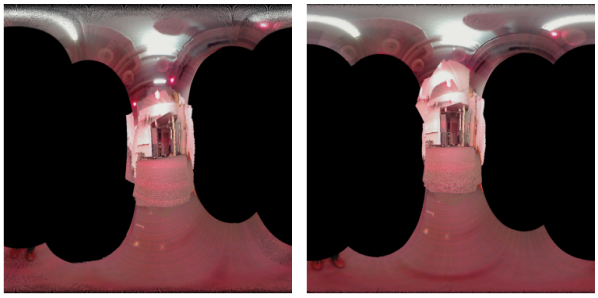


Fig. 7. Comparing the IMU formed panorama with the VICON ground truth panorama for training set 2

similar. The minor distortion around the edges within the IMU images is caused by the slight jitter which takes place due to the linear change of gradient descent.

The black spaces in the panorama are caused by the camera not visiting that specific orientation/area, so we do not have a view corresponding with the area, resulting in us leaving it blank.

2) Test Data: We display the results of creating a panorama from our orientations after gradient descent in Fig 8 and 9, and although some orientation values went to nan, we see that the constructed panorama is actually very cohesive and looks similar to our training set panoramas, where we can see and understand details within the image of the hallway and the surroundings.

However, in the first panorama made from test set 10, we can see significant jitter. This is due to the fact that the values for pitch and roll stayed almost exactly the same throughout the period, and the gradient descent step size was too high, resulting in a noisy gradient descent result.



Fig. 8. Displaying the panorama formed using the orientations and images in test set 10



Fig. 9. Displaying the panorama formed using the orientations and images in test set 11

V. Conclusion

We present an algorithm to solve for more accurate orientation reconstruction on IMU data, using gradient descent methods and a physics-based problem formulation. We then apply this method to creating a panorama from camera images, by using timestamp matching along with cylindrical projection to form accurate panoramas of videos taken with a moving camera. This algorithm is a lightweight solution to the problem of using IMUs along with cameras, and to use IMUs in general while giving accurate results. This report also shows the way forward, in terms of improving gradient descent methods and the potential for other visual methods to improve scene reconstruction problems.

References

I only referenced random online articles for the conversion from spherical to cyindrical and cartesian frames, and Prof Atanasov's in class notes.

Appendix A

Appendix B

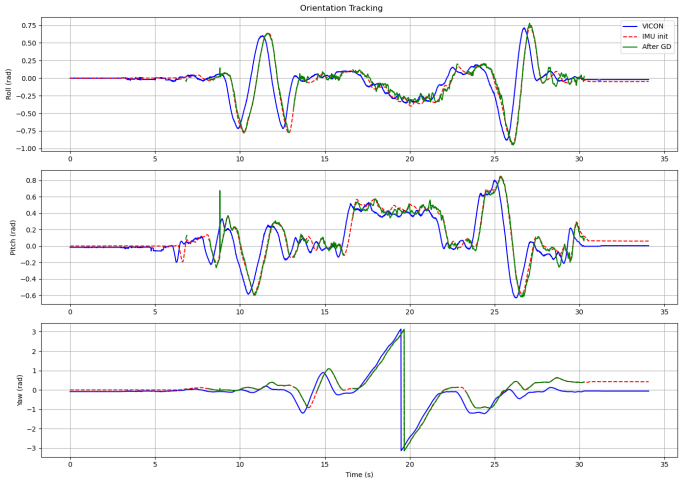


Fig. 10. Training gradient descent parameters for training set 3, and comparing the results with the ground truth orientations

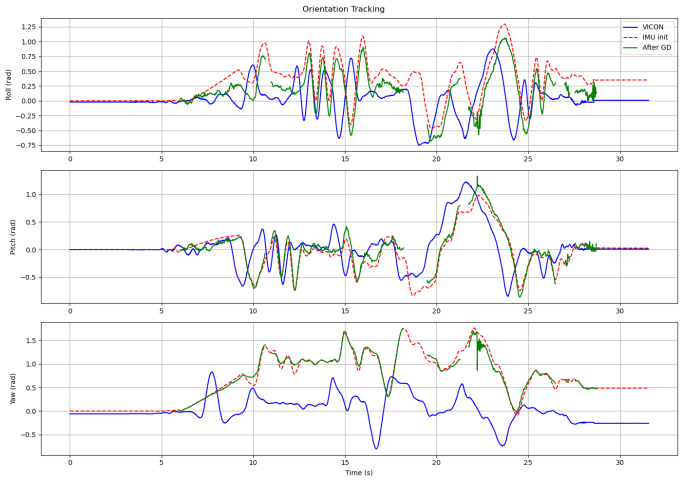


Fig. 11. Training gradient descent parameters for training set 4, and comparing the results with the ground truth orientations

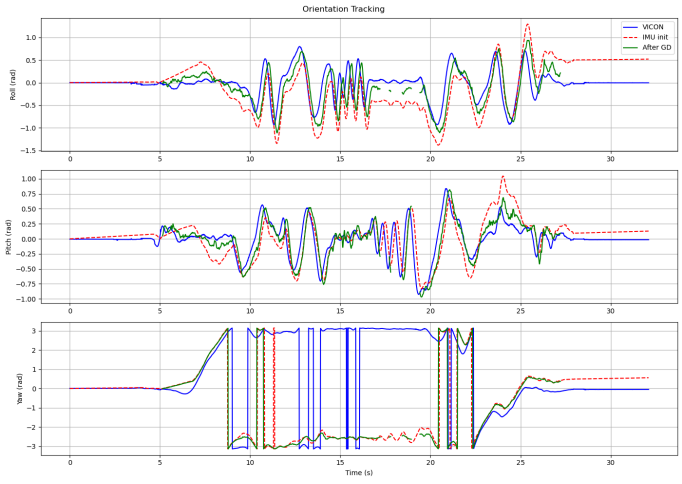


Fig. 12. Training gradient descent parameters for training set 5, and comparing the results with the ground truth orientations

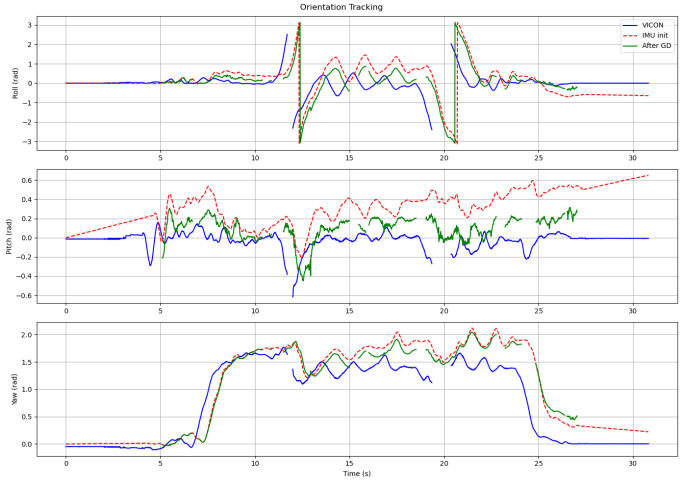


Fig. 13. Training gradient descent parameters for training set 6, and comparing the results with the ground truth orientations

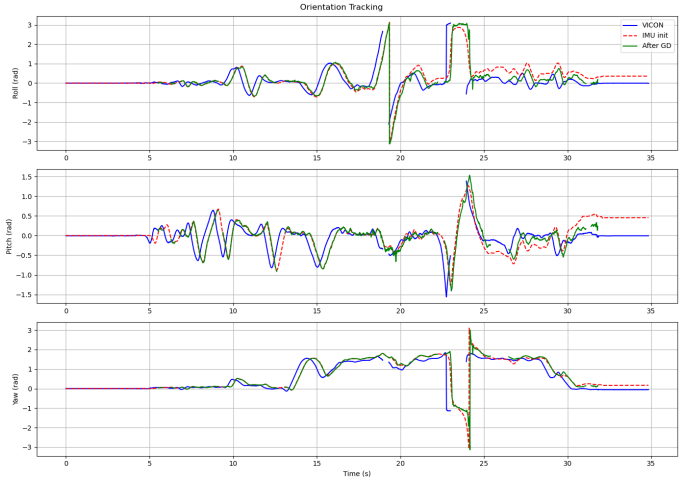


Fig. 14. Training gradient descent parameters for training set 7, and comparing the results with the ground truth orientations

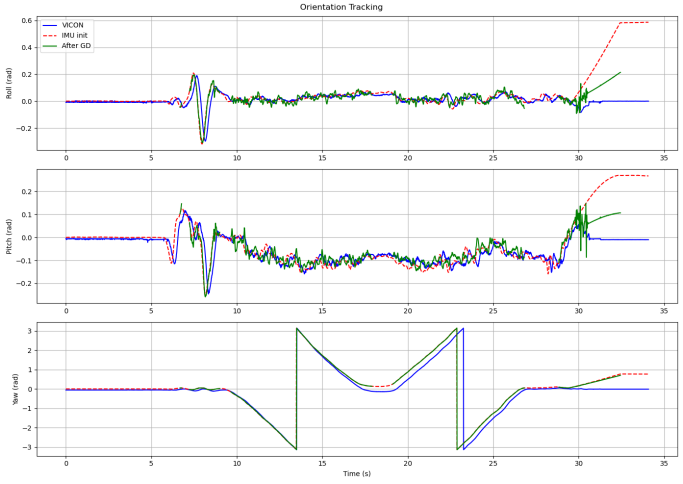


Fig. 15. Training gradient descent parameters for training set 8, and comparing the results with the ground truth orientations

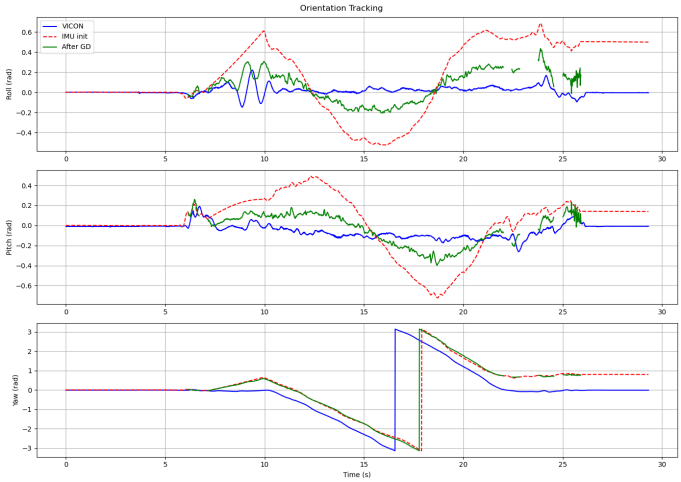


Fig. 16. Training gradient descent parameters for training set 9, and comparing the results with the ground truth orientations

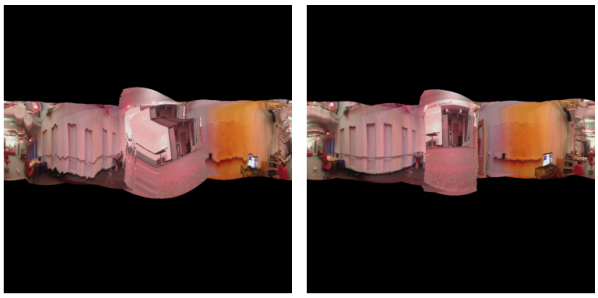


Fig. 17. Comparing the IMU formed panorama with the VICON ground truth panorama for training set 8

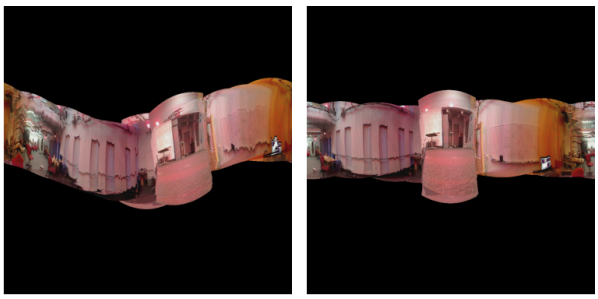


Fig. 18. Comparing the IMU formed panorama with the VICON ground truth panorama for training set 9